

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
КАФЕДРА ИНТЕЛЛЕКТУАЛЬНЫХ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

ОТЧЁТ

по лабораторной работе №7

Выполнил:

студент группы ПО-12

Снарский П.П.

Проверила:

Кулик А.Д.

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Вариант 12

Ход работы

12) Задать составление строки из символов, появляющихся из разных углов формы и выстраивающихся друг за другом. Процесс должен циклически повторяться.

12) Кривая Гильберта

```
"""Lab7 module"""
import tkinter as tk
from PIL import ImageGrab
import math
import time

class FlyingSymbol:
    """Flying symbol class"""
    def __init__(
        self, canvas, char, start_x, start_y, target_x, target_y, speed=5,
        color="blue"
    ):

        self.canvas = canvas
        self.char = char

        self.x = start_x
        self.y = start_y

        self.target_x = target_x
        self.target_y = target_y

        self.speed = speed

        self.obj = canvas.create_text(
            self.x, self.y, text=self.char, fill=color, font=("Arial", 24,
"bold")
        )

    def move(self):
        """move symbol function"""
        dx = self.target_x - self.x
        dy = self.target_y - self.y

        dist = math.sqrt(dx * dx + dy * dy)

        if dist < self.speed:
            self.canvas.coords(self.obj, self.target_x, self.target_y)
            return True

        self.x += self.speed * dx / dist
        self.y += self.speed * dy / dist

        self.canvas.coords(self.obj, self.x, self.y)

        return False
```

```

class StringAnimation:
    """String animation class"""

    def __init__(self, canvas):
        self.canvas = canvas
        self.symbols = []
        self.running = False

    def start(self, text, speed):
        """Start animation func"""
        self.canvas.delete("all")
        self.symbols.clear()

        width = self.canvas.winfo_width()
        height = self.canvas.winfo_height()

        center_y = height // 2

        start_positions = [(0, 0), (width, 0), (0, height), (width, height)]

        spacing = 40
        start_x = width // 2 - (len(text) * spacing) // 2

        for i, ch in enumerate(text):
            sx, sy = start_positions[i % 4]

            tx = start_x + i * spacing
            ty = center_y

            symbol = FlyingSymbol(self.canvas, ch, sx, sy, tx, ty,
speed=speed)

            self.symbols.append(symbol)

        self.running = True
        self.animate()

    def animate(self):
        """Animate func"""

        if not self.running:
            return

        finished = True

        for s in self.symbols:
            done = s.move()

            if not done:
                finished = False

        if finished:
            self.canvas.after(1000, self.restart)
        else:
            self.canvas.after(20, self.animate)

    def restart(self):
        """Restart func"""
        if self.running:
            app.start_animation()

    def stop(self):
        """Stop func"""
        self.running = False

```

```

class HilbertCurve:
    """Hilbert fractal class"""

    def __init__(self, canvas):
        self.canvas = canvas
        self.points = []

    def draw(self, order, size):
        """Draw func"""

        self.canvas.delete("all")

        self.points.clear()

        self.x = 50
        self.y = 50

        self.angle = 0

        self.step = size / (2**order)

        self.points.append((self.x, self.y))

        self.hilbert(order, 90)

        for i in range(len(self.points) - 1):
            x1, y1 = self.points[i]
            x2, y2 = self.points[i + 1]

            self.canvas.create_line(x1, y1, x2, y2, fill="darkgreen",
width=2)

    def forward(self):
        """Forward func"""
        rad = math.radians(self.angle)

        self.x += self.step * math.cos(rad)
        self.y += self.step * math.sin(rad)

        self.points.append((self.x, self.y))

    def left(self, angle):
        """Left func"""
        self.angle += angle

    def right(self, angle):
        """Right func"""
        self.angle -= angle

    def hilbert(self, level, angle):
        """Hilbert main func"""

        if level == 0:
            return

        self.right(angle)
        self.hilbert(level - 1, -angle)

        self.forward()

        self.left(angle)
        self.hilbert(level - 1, angle)

```

```

        self.forward()

        self.hilbert(level - 1, angle)

        self.left(angle)
        self.forward()

        self.hilbert(level - 1, -angle)
        self.right(angle)

class Application:
    """Main window class"""

    def __init__(self, root):

        self.root = root
        self.root.title("Лабораторная работа")

        control = tk.Frame(root)
        control.pack(side=tk.TOP, fill=tk.X)

        tk.Label(control, text="Стропока:").pack(side=tk.LEFT)

        self.text_entry = tk.Entry(control, width=20)
        self.text_entry.insert(0, "HELLO")
        self.text_entry.pack(side=tk.LEFT, padx=5)

        tk.Label(control, text="Скорость:").pack(side=tk.LEFT)

        self.speed_scale = tk.Scale(control, from_=1, to=20,
orient=tk.HORIZONTAL)
        self.speed_scale.set(5)
        self.speed_scale.pack(side=tk.LEFT)

        self.start_btn = tk.Button(control, text="Старт",
command=self.start_animation)
        self.start_btn.pack(side=tk.LEFT, padx=5)

        self.pause_btn = tk.Button(control, text="Пауза",
command=self.pause_animation)
        self.pause_btn.pack(side=tk.LEFT, padx=5)

        self.screenshot_btn = tk.Button(
            control, text="Скриншот", command=self.make_screenshot
        )
        self.screenshot_btn.pack(side=tk.LEFT, padx=5)

        fractal_frame = tk.Frame(root)
        fractal_frame.pack(side=tk.TOP, fill=tk.X)

        tk.Label(fractal_frame, text="Порядок Гильберта:").pack(side=tk.LEFT)

        self.order_scale = tk.Scale(fractal_frame, from_=1, to=6,
orient=tk.HORIZONTAL)
        self.order_scale.set(4)
        self.order_scale.pack(side=tk.LEFT)

        self.fractal_btn = tk.Button(
            fractal_frame, text="Построить фрактал",
command=self.draw_hilbert
        )
        self.fractal_btn.pack(side=tk.LEFT, padx=5)

```

```

        self.canvas = tk.Canvas(root, width=900, height=600, bg="white")

        self.canvas.pack(fill=tk.BOTH, expand=True)

        self.anim = StringAnimation(self.canvas)
        self.hilbert = HilbertCurve(self.canvas)

    def start_animation(self):
        """Start animation func"""

        text = self.text_entry.get()

        if text.strip() == "":
            return

        speed = self.speed_scale.get()

        self.anim.stop()

        self.anim = StringAnimation(self.canvas)

        self.anim.start(text, speed)

    def pause_animation(self):
        """Pause animation func"""

        if self.anim.running:
            self.anim.stop()
            self.pause_btn.config(text="Продолжить")
        else:
            self.start_animation()
            self.pause_btn.config(text="Пауза")

    def make_screenshot(self):
        """Screenshot func"""

        x = self.root.winfo_rootx()
        y = self.root.winfo_rooty()

        x1 = x + self.root.winfo_width()
        y1 = y + self.root.winfo_height()

        img = ImageGrab.grab().crop((x, y, x1, y1))

        filename = f"screenshot_{int(time.time())}.png"

        img.save(filename)

        print("Скриншот сохранен:", filename)

    def draw_hilbert(self):
        """Drawing hilbert curve"""

        order = self.order_scale.get()

        self.hilbert.draw(order, 500)

root = tk.Tk()

app = Application(root)

root.mainloop()

```

Вывод: в ходе выполнения лабораторной работы освоил возможности языка программирования Python в разработке оконных приложений